

Template Matching

Template Matching

Preface

Example Results

Principle Explanation

What is Template Matching

Response Map and Best Position

Code Overview

Initialize Camera and Display Module

Create Template and Threshold

Downsample and Binarize

Match and Find Best Position

Draw Match Box

Print Score and Release Resources

Parameter Adjustment Notes

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. You can download it from the Jianan website https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/

 **K230 MicroPython 镜像下载**

CI/CD 更新时间: 2026-07-28 01:29:59

选择开发板

 CanMV K230 V1.0/V1.1

 CanMV K230 V3.0

 庐山派 K230

 庐山派 Lite K230D

 **Yahboom K230**

 01Studio K230

 01Studio K230 Emmc

 Hiwonder K230

 WonderMK K230

 GT6700 K230

 **勘智 KENDRYTE**

Yahboom K230
MicroPython Build for Yahboom K230

当前下载镜像:
[CanMV_K230_YAHBOOM_micropython_v1.8-28-g9b151af_nncase_v2.11.0.img.gz](#)

最新每日构建版本
Daily-0728
编译日期: 2026-07-28

MD5: 66f0622fc6b8d22e55f552a40458d2fa
[立即下载](#)

极简 USB 烧录指南 (推荐)

 镜像仅可烧录到对应的开发板, 请务必选择与当前板型完全匹配的镜像, 不能混烧其他板型的镜像。否则可能导致硬件损坏!!!

1 准备工具
下载并打开 [K230BurningTool](#) 烧录工具, 并加载固件。

2 进入烧录模式 (根据硬件情况选择)

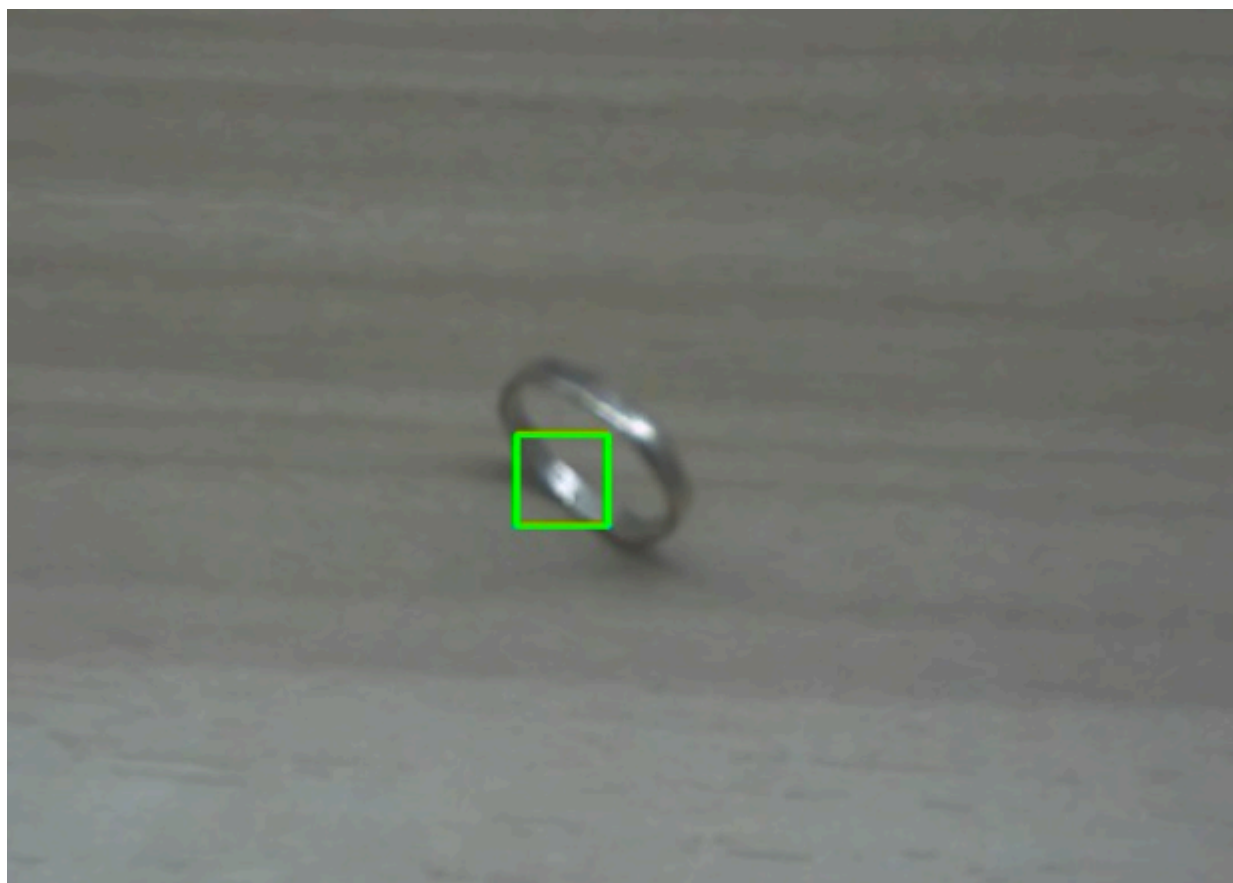
☒ 有 BOOT 键的板子

☐ 无 BOOT 键的板子

Example Results

Run the example code for this section [09.camera_match_template.py](#).

This section downsamples the live image and applies binarization, uses a fixed 20×20 template for normalized correlation coefficient matching, finds the highest score position via `minMaxLoc()`, and draws the matching box back onto the original image.



Principle Explanation

What is Template Matching

Template matching slides a small image pixel by pixel across the search image and calculates a similarity score for each position. `TM_CCOEFF_NORMED` returns the normalized correlation coefficient; the score closer to 1 means the template is more similar to that region.

Template matching is sensitive to scale, rotation, perspective, and appearance changes. The example uses a fixed binary template, suitable for demonstrating the API, but does not represent the ability to recognize arbitrary white targets.

Response Map and Best Position

If the search image size is $W \times H$, the template size is $w \times h$, the response map size is typically:

$$(W - w + 1) \times (H - h + 1)$$

`minMaxLoc()` returns the minimum value, maximum value, and their positions. For `TM_CCOEFF_NORMED`, this example uses the maximum value position as the best match point.

Code Overview

Initialize Camera and Display Module

```
W, H = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_id=True)
sensor.run()
time.sleep(0.5)
```

Create Template and Threshold

```
TS = 20
templ = np.zeros((TS, TS, 1), dtype=np.uint8)
cv2.rectangle(templ, (8, 8), (TS - 8, TS - 8), (255,), -1)
THR = 0.6
```

The template outer frame size is 20×20, with a white area drawn from coordinates 8 to 12. The "40×40 white square template" in the source code comments refers to the template outer frame description after mapping to the original image, not that the template array itself is 40×40.

Downsample and Binarize

```
small = cv2.resize(img_np, (320, 240))
gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray, 200, 255, cv2.THRESH_BINARY)
```

The fixed threshold of 200 only keeps brighter regions, making the search image data format consistent with the black background white block template.

Match and Find Best Position

```
response = cv2.matchTemplate(
    binary,
    templ,
    cv2.TM_CCORR_NORMED
)
min_value, max_value, min_location, max_location = cv2.minMaxLoc(response)
```

max_value is the best score, and max_location is the coordinates of the template's top-left corner in the 320×240 search image.

Draw Match Box

```
if max_value > THR:
    x = max_location[0] * 2
    y = max_location[1] * 2
    cv2.rectangle(
        img_np,
        (x, y),
        (x + TS * 2, y + TS * 2),
        (0, 255, 0), 2
    )
```

Both template size and position are multiplied by 2 to map the 320×240 match result to the 640×480 image.

Print Score and Release Resources

```
if time.time_diff(time.time_ms(), ft) >= 1000:
    print("match best:{:.2f} | fps:{}".format(max_value, fc))

finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Template Content:** The template should come from real targets processed through the same preprocessing pipeline. A fixed hand-drawn template is only suitable for API demonstration.
- **Threshold 200:** Lower it when the target is not bright enough; raise it when background bright areas cause false detections. The template and search image must use consistent binarization rules.
- **Confidence THR=0.6:** Raise it when false detections are high; lower it when missed detections are high. Determine a reasonable range through positive and negative sample statistics.
- **Scale Variation:** When target size varies, a single template can easily fail. Prepare multiple scale templates or search within a limited scale range.
- **Rotation Variation:** Template matching does not have rotation invariance inherently. You need to prepare rotated templates or switch to feature-based methods.
- **Search Region:** Limiting ROI can significantly reduce computation and background false detections, making it the preferred optimization for embedded real-time applications.
- **Template Size:** The template must not be larger than the search image; otherwise, the function cannot produce a valid response map.